pi-billboard: Audience-Aware Digital Signage on a Raspberry Pi

Dilan Perera *ISMR Labs*
`https://github.com/ISMR-Labs/pi-billboard`

October 2026

Abstract

Digital billboards usually play a fixed loop regardless of who is watching. *pi-billboard* is a self-contained signage platform that runs entirely on a Raspberry Pi with a camera. Ads are uploaded into categories through a web dashboard; during playback the camera counts the people in front of the screen and, from a lightweight head-pose estimate, how many of them are looking at it. The attention rate of each category (lookers / viewers) drives a Thompson-sampling scheduler with time decay, so the screen converges on the content its current audience responds to while still exploring. No cloud service, GPU or identity data is needed: only counts leave the camera thread. The two vision models total under 5 MB and run at 5–8 fps on a Pi 4.

1 Introduction

Attention is the scarce resource in out-of-home advertising, yet small-scale signage cannot measure it and therefore cannot act on it. Commercial audience-measurement products depend on cloud analytics, dedicated hardware or face recognition, which raises cost and privacy concerns. Our goal was a system a shop, lab or campus can deploy on a $50 computer, that keeps all processing on the device, and that *closes the loop*: measurement changes what is played instead of only producing a report.

The platform makes three contributions. (1) An on-device audience sensor that separates *presence* from *attention* by estimating head orientation relative to the screen. (2) A tracker that turns per-frame detections into per-person counts, so a viewer is counted once per ad rather than once per frame. (3) A scheduler that treats the choice of category as a multi-armed bandit, handling the explore/exploit trade-off and a non-stationary audience (lunch crowd versus evening crowd) in a principled way. Around this sit the practical parts: upload and categorisation of image and video ads, dayparting, a live dashboard with per-ad and per-category statistics, and a kiosk player for the display (Fig. 1).

2 Methodology

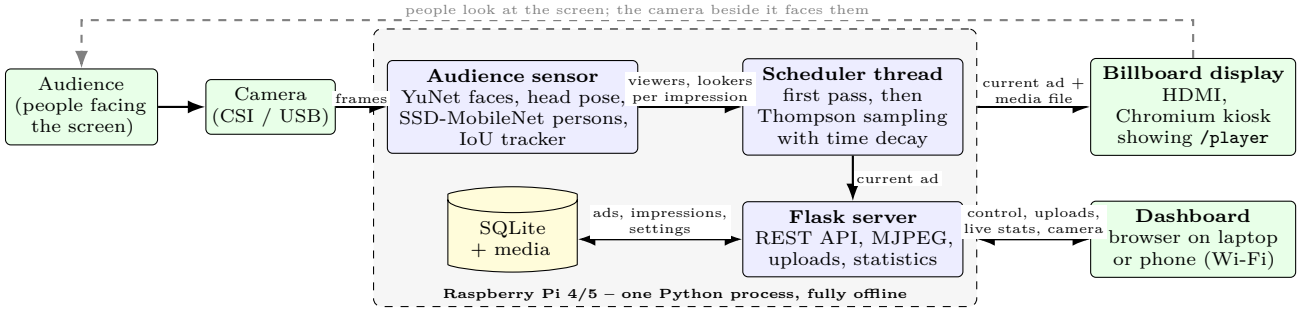


Figure 1: Overall architecture. The camera sits beside the display and faces the audience; the display is a browser in kiosk mode that polls the server for the current ad. Everything runs offline on the Pi.

Audience sensing. Each frame is processed by two small models: YuNet (OpenCV, 230 KB) returns face boxes with five landmarks, and a quantised SSD-MobileNet v1 (TFLite, 4 MB, run every third frame) returns person boxes so that people facing away are still counted. From the five landmarks we solve a perspective-n-point problem against a generic 3-D face model to obtain the head's rotation R and translation t in camera coordinates. The attention angle is $\theta = \arccos(\hat{f} \cdot \hat{c})$ with $\hat{f} = R\,[0, 0, -1]^\top$ the head's forward direction and $\hat{c} = -t/\|t\|$ the direction from the head to the camera; a face is *looking* when $\theta < 25°$. Using the direction to the camera rather than raw yaw classifies a person at the edge of the frame who turns toward the screen correctly. Faces are merged into person boxes by containment and a greedy IoU tracker assigns stable ids. During an ad play (an *impression*) a track is a *viewer* if present for ≥ 0.5 s and a *looker* if it faced the screen for ≥ 0.7 s, giving per-person rather than per-frame counts.

Scheduling. Let category c have $v_{c,i}$ viewers and $\ell_{c,i}$ lookers in its past impression i. We keep a Beta belief over its attention rate, $p_c \sim \mathrm{Beta}\big(1 + \sum_i w_i\,\ell_{c,i},\ 1 + \sum_i w_i\,(v_{c,i} - \ell_{c,i})\big)$, with $w_i = 2^{-(t_{\mathrm{now}} - t_i)/h}$ discounting old impressions by a half-life h (default 2 h) so the belief follows a changing audience. When playback starts, one ad from every category is shown. Thereafter, before each slot, a sample $\tilde{p}_c$ is drawn per category and the arg-max is played (Thompson sampling); with probability $\varepsilon = 0.1$ a random category is played so none starves. The same sampling picks the ad within the category, excluding the one that just played. Impressions with no viewers do not update the belief, and categories added at runtime get an immediate first play.

Platform. A Flask server with SQLite exposes a REST API, an MJPEG camera stream, a dashboard (categories, uploads, start/stop/skip, per-ad and per-category plays, viewers, lookers, attention and current belief, hourly chart, CSV export) and the player page that Chromium shows fullscreen on the Pi's HDMI output. Media and data stay on the Pi, no internet access is required, and playback resumes after a reboot.